

CityTech TTP Summer Bootcamp

Mini Workshop: Authentication

2026-08-10

Agenda

1. Why authentication
2. Hashing passwords safely
3. Cookies vs JWTs
4. Passport.js and protected routes
5. Aside: forgot password
6. The login flow, end to end
7. Client: TanStack Query and the token
8. Logging out with Zustand

Why Authentication

Your server handles requests from many different people. It needs to know **who is asking**.

- **Authentication:** verifying **who you are** (log in, get proven identity back)
- **Authorization:** deciding **what you're allowed to do**, once your identity is known
- Authentication comes first: you can't authorize an identity you don't know
- Today: how a browser or mobile app proves identity to an Express and PostgreSQL backend, and keeps proving it on every later request

Encryption vs Hashing

Two very different operations, easy to mix up:

- **Encryption** is **two-way**: reversible with the right key. Encrypt, then decrypt, and you get the original back
- **Hashing** is **one-way**: a fixed-length "fingerprint" of the input. There is no key that turns a hash back into the original
- The same input always hashes to the same output; changing even one character produces a completely different hash

For passwords, we want one-way: even if the database leaks, the actual password should never be recoverable.

Encryption vs Hashing: Why It Matters

- If passwords were **encrypted**, whoever holds the decryption key could read every user's real password
- If passwords are **hashed**, there is no key to steal. The server never even needs to know your real password again after signup
- Login just re-hashes what you typed and **compares hashes** - never "decrypts"
- Rule of thumb: encrypt data you need to read again later (like a credit card in a payment flow); hash data you only ever need to *verify* (like a password)

The Threat: Hashes Can Be Broken Over Time

A leaked table of hashes is not automatically safe. Attackers work backward from the hash:

- **Rainbow tables:** huge precomputed lookup tables of common passwords and their hashes - instant reversal for anything common
- **Brute force:** try every possible password, hash each one, compare. Slow in 2010, much faster today with GPUs and specialized ASIC hardware
- A hashing algorithm that was "secure enough" a decade ago can become crackable as hardware and technique improve
- This is an arms race, not a one-time decision

The Threat: Defenses

Salting: mix a random value into each password before hashing, so identical passwords produce different hashes and precomputed rainbow tables stop working.

```
await bcrypt.hash("hunter2", 10); // "$2b$10$N9qo8uL0ickgx2ZMRZoMye7NRfHM3aRfxUq7NcQczX2vT1u5jUY0e"  
await bcrypt.hash("hunter2", 10); // "$2b$10$K3fjq7HQzr8wXvHmZDl.90eD1zjX6uXHqU8u/M6zn6d0S8gzYFhVy"  
// same password, different hash – bcrypt generates and embeds a random salt for you
```

The salt isn't stored separately: it's embedded in the hash string itself (right after the cost factor). So `bcrypt.compare()` never needs a new random salt – it reads the salt back out of the stored hash, re-hashes the login attempt with that same salt, and compares.

Storing Passwords Safely: Code

```
// prisma/schema.prisma
model User {
  id          Int    @id @default(autoincrement())
  email       String @unique
  passwordHash String
}
```

```
// signup: hash before saving, never store the raw password
import bcrypt from "bcrypt";

const passwordHash = await bcrypt.hash(req.body.password, 10); // 10 = work factor
await prisma.user.create({ data: { email: req.body.email, passwordHash } });

// login: compare, never decrypt
const user = await prisma.user.findUnique({ where: { email: req.body.email } });
const valid = user && (await bcrypt.compare(req.body.password, user.passwordHash));
```

The Need for Something Client Side

Passwords get a user **logged in** for one request. But HTTP is **stateless**: the very next request looks like a brand new stranger.

- The server does not remember you between requests on its own
- Asking for a password on **every single request** would be unusable
- So after a successful login, the server hands the client something it can **send back on every later request** as proof: "I already proved who I am"
- Two industry-standard ways to carry that proof: **session cookies** and **JSON Web Tokens (JWT)**

Session Cookies

- After login, the server creates a **session record** (often in a database or in-memory store) and gives it an ID
- That ID is sent to the browser as a **cookie**; the browser **automatically re-sends it** on every request to that domain
- The server looks up the session ID on each request to find out who's asking
- **Industry standard**: mark the cookie `HttpOnly` (JavaScript can't read it, blocking a class of XSS attacks), `Secure` (only sent over HTTPS), and `SameSite` (limits which sites can trigger it)
- The server holds the state: revoking a session just deletes that record

JSON Web Tokens (JWT)

- After login, the server **signs** a token containing the user's identity (for example their user ID) and hands it directly to the client
- The client stores the token itself and **manually attaches it** to every later request, typically as a header: `Authorization: Bearer <token>`
- The server does not store anything: it just **verifies the signature** on each request. Anyone holding a valid signature is trusted
- **Industry standard: JWT**, defined by **RFC 7519** - a signed, self-contained token, not encrypted (its contents are readable, just tamper-proof)
- Nothing to look up server side, but also nothing to easily revoke early: a JWT is valid until it expires

Comparing Cookies vs JWTs

Session cookies

- Server-side **state**: a session store to maintain
- Browser attaches the cookie **automatically**
- Revoke instantly: delete the session record

JWTs

- **Stateless**: nothing stored server side
- Client must **manually attach** the header
- Can't easily revoke before it expires
- Works anywhere you can set a header

Ionic apps run in a **Capacitor** native shell, not always a full browser cookie jar - so **token-based auth is the common choice** for mobile and API-first apps like our capstone stack.

Passport.js: The Industry-Standard Middleware

Passport is Express's industry-standard authentication middleware. Instead of writing auth logic from scratch, you plug in a **strategy** for however you want to verify identity.

- **passport-local** : verifies a username and password at login (using `bcrypt.compare` under the hood)
- **passport-jwt** : verifies a bearer token on protected routes
- Same pattern, swappable strategy: the same app could add **passport-google-oauth20** later without changing how routes are protected
- Docs: passportjs.org

Login Route: Code

```
// passport-local strategy: verify email + password
passport.use(new LocalStrategy({ usernameField: "email" }, async (email, password, done) => {
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return done(null, false, { message: "Invalid credentials" });
  }
  return done(null, user);
}));

// POST /login: authenticate, then sign and return a JWT
app.post("/login", passport.authenticate("local", { session: false }), (req, res) => {
  const token = jwt.sign({ sub: req.user.id }, process.env.JWT_SECRET, { expiresIn: "1h" });
  res.json({ token });
});
```

Protecting Routes: Middleware

```
// passport-jwt strategy: verify the bearer token's signature
passport.use(new JwtStrategy({
  jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  secretOrKey: process.env.JWT_SECRET,
}, async (payload, done) => {
  const user = await prisma.user.findUnique({ where: { id: payload.sub } });
  return done(null, user ?? false);
}));

// GET /me: protected route, only reachable with a valid token
app.get("/me", passport.authenticate("jwt", { session: false }), (req, res) => {
  res.json({ id: req.user.id, email: req.user.email });
});
```

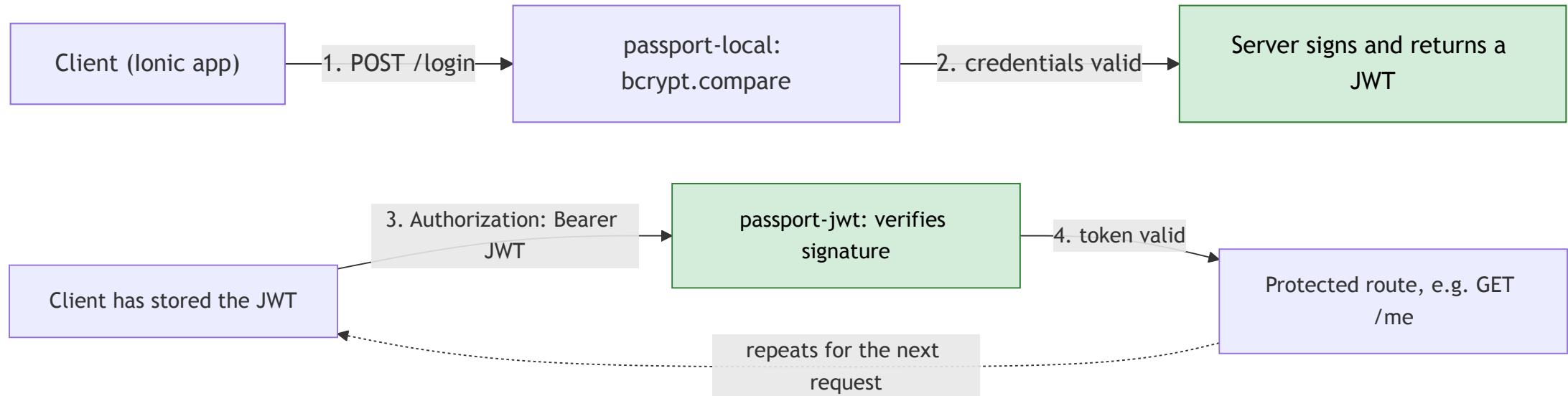
`{ session: false }` opts out of Passport's cookie-session mode - we're staying fully token based.

Aside: Forgot Password

Knowing someone's email is not proof you *are* them, so "forgot password" needs its own identity check before it can touch a password.

- The standard pattern is a **second channel**: email them a one-time link (or code), and only let the reset happen through that link
- That link usually carries a **short-lived, single-use token**, separate from a login JWT, tied to one user and one reset attempt
- This is effectively a lightweight form of **two-factor verification**: something you know (your email account) proving you can also act on the account you're resetting
- **Out of scope for today**: it needs its own token type, expiry, and (usually) an email-sending service - a real feature to design and build separately, not an add-on to the login flow above

The Login Flow, End to End



Client: TanStack Query - Login

```
// useLogin.js – a mutation, not a query: login changes server state
function useLogin() {
  return useMutation({
    mutationFn: (credentials) =>
      fetch(`${API_URL}/login`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(credentials),
      }).then((res) => res.json()),
    onSuccess: ({ token }) => localStorage.setItem("token", token),
  });
}
```

`useLogin` is a **mutation**, not a query: it changes state on the server (and now the client), rather than just reading it.

Client: TanStack Query - Using the Token

```
// useMe.js - attach the stored token to every later request
function useMe() {
  const token = localStorage.getItem("token");
  return useQuery({
    queryKey: ["me"],
    queryFn: () =>
      fetch(`${API_URL}/me`, { headers: { Authorization: `Bearer ${token}` } }).then((res) => res.json()),
    enabled: !!token,
  });
}
```

Either storage works: raw `localStorage` (above) is the simplest option, or a **Zustand** store with the **persist** middleware - same persistence, but the token becomes reactive state any component can subscribe to, no manual re-render needed. The demo uses the Zustand version.

Client: Zustand - Logging Out

```
// useAuthStore.ts - clearToken resets the store back to null
const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      setToken: (token) => set({ token }),
      clearToken: () => set({ token: null }),
    }),
    { name: "auth-demo-token" }
  )
);
```

- Logging out is just **resetting state** - no server round trip needed, since our JWT is stateless
- Any component can call `clearToken()` : the store updates, `persist` mirrors the change to `localStorage`, and every subscribed component re-renders

Follow Up

- **Hash, don't encrypt** passwords - use bcrypt or Argon2
- HTTP is stateless: carry proof of identity after login via **cookies** or **JWTs**
- Our capstone stack favors **JWTs** with **Passport.js** (`passport-local` + `passport-jwt`), since a Capacitor app can't rely on a browser cookie jar
- A full runnable example lives in `demos/auth-demo` - register, log in, fetch a protected profile, and log out, wired up end to end with Express, Prisma, and an Ionic React client
- Docs: [Passport.js](#), [jsonwebtoken](#), [OWASP Password Storage Cheat Sheet](#)

Let's Build It

demos/auth-demo

Register, log in, fetch a protected profile, and log out - Express, Prisma, and Passport on the server, Ionic React and TanStack Query on the client, wired up end to end.

Follow the README and try it yourself.